

A taxonomy version stops being *a container of rows* and becomes *a marker in time*. Nothing is copied. The question "what wording was in force for this run?" is answered by resolving as of a date, not by looking up a snapshot.

That needs three things: a change log that records only what actually changed, a release marker with effective dates, and — the important one — a record of the exact payload sent to the models.

1. Change log, not snapshots

Leave the existing dimension as the current state. Add an append-only history table beside it that captures only changed rows:

```
sql
CREATE TABLE validation.skill_definition_history (
  history_id    BIGINT IDENTITY PRIMARY KEY,
  skill_code    VARCHAR(20) NOT NULL REFERENCES validation.skill(skill_code),
  valid_from    DATETIME2(0) NOT NULL,
  valid_to      DATETIME2(0) NULL,      -- NULL = currently in force
  definition_text NVARCHAR(MAX) NOT NULL,
  inclusions    NVARCHAR(MAX) NULL,
  exclusions    NVARCHAR(MAX) NULL,
  content_hash  BINARY(32) NOT NULL,    -- SHA2_256 of normalised text
  change_source VARCHAR(20) NOT NULL    -- validation / upstream / correction
    CHECK (change_source IN ('validation','upstream','correction')),
  change_id     VARCHAR(20) NULL        -- the request that caused it, if ours
    REFERENCES validation.definition_change(change_id),
  recorded_by   VARCHAR(20) NULL,
  recorded_at   DATETIME2(0) NOT NULL DEFAULT SYSUTCDATETIME()
);
```

```
CREATE UNIQUE INDEX uq_sdh_current ON validation.skill_definition_history
(skill_code)
```

```
WHERE valid_to IS NULL;
```

```
CREATE INDEX ix_sdh_asof ON validation.skill_definition_history (skill_code,
valid_from, valid_to);
```

One row per *actual change* to one skill. If three skills are reworded, that's three rows, not sixty.

content_hash is what makes this maintainable. A nightly job hashes the current dimension rows and compares against the open history row. If the hash differs, close the old row and insert a new one. No triggers on a table you don't own, and it catches changes made upstream by someone else, which is the scenario most likely to break your results quietly.

change_source matters for the same reason. A definition that changes because another team edited the dimension is a very different event from one you requested, and you want to see it in the report.

2. Release marker with dates

sql

-- taxonomy_version becomes a marker, not a container

```
ALTER TABLE validation.taxonomy_version ADD effective_from DATETIME2(0) NOT NULL;
```

```
ALTER TABLE validation.taxonomy_version ADD effective_to DATETIME2(0) NULL;
```

TAX-1.1 now means "the state of all definitions from 18 May onward". It holds no definition rows at all.

3. The payload record — the piece I'd insist on

This is what actually closes the loop. Whatever the dimension says, record exactly what was handed to the data science team for each run:

sql

```
CREATE TABLE validation.definition_package (  
    package_id VARCHAR(20) PRIMARY KEY,  
    run_id VARCHAR(20) NULL REFERENCES validation.model_run(run_id),  
    created_at DATETIME2(0) NOT NULL,  
    skill_count INT NOT NULL,  
    payload_hash BINARY(32) NOT NULL, -- hash of the whole package  
    payload NVARCHAR(MAX) NULL, -- the JSON actually sent  
    sent_to NVARCHAR(120) NULL  
);
```

```
CREATE TABLE validation.definition_package_line (  
    package_id VARCHAR(20) NOT NULL REFERENCES  
validation.definition_package(package_id),  
    skill_code VARCHAR(20) NOT NULL REFERENCES validation.skill(skill_code),  
    history_id BIGINT NOT NULL REFERENCES  
validation.skill_definition_history(history_id),  
    PRIMARY KEY (package_id, skill_code)  
);
```

The package lines are pointers into the history, so you store the wording once but can always reconstruct the exact text any run saw. If someone asks "did the model actually get the new firmware definition?", that's one join, not an archaeology exercise.

It also catches the failure mode where definitions were updated in the dimension but the extract to data science ran before the update. Without the package record, that looks like a model failure.

The lineage chain

That gives you an unbroken path in both directions:

review_verdict (SME said definition gap)

- definition_change (reword request, approved)
- skill_definition_history (the new wording, valid_from)
- definition_package_line
- definition_package → model_run
- model_output (what the model produced with it)
- auto_assessment → review_verdict (next cycle)

So you can ask either "which cases led to this wording change?" or "did accuracy on this skill improve after the change?" and both are answerable with joins rather than judgement.

Resolving the definition in force

```
sql
-- What a model saw for a run (preferred: from the package)
SELECT h.skill_code, h.definition_text, h.inclusions, h.exclusions
FROM validation.definition_package p
JOIN validation.definition_package_line l ON l.package_id = p.package_id
JOIN validation.skill_definition_history h ON h.history_id = l.history_id
WHERE p.run_id = @run_id;

-- Fallback where no package exists (historic runs): as-of resolution
SELECT h.skill_code, h.definition_text
FROM validation.model_run r
JOIN validation.skill_definition_history h
  ON r.run_date >= h.valid_from
  AND (h.valid_to IS NULL OR r.run_date < h.valid_to)
WHERE r.run_id = @run_id;
```

Wrap the first in a view, say `v_run_definitions`, so the form and reporting both use one definition of "the wording for this run" rather than each re-deriving it.

What this changes elsewhere

Where	Change
<code>skill_definition</code>	Drop the table. The upstream dimension plus the history table replace it.
<code>model_run.taxonomy_version</code>	Keep it as a human-readable label, but resolution happens via <code>definition_package</code> .
Form spec	"Filter <code>skill_definition</code> on the run's taxonomy version" becomes "read <code>v_run_definitions</code> for the queue row's <code>run_id</code> ".
Data load checklist	Step 2 no longer loads definitions per version. It seeds the history table with the opening state of each skill.

The trade-off, stated plainly

As-of resolution is slightly harder to query than a snapshot, and a careless join can return two rows for one skill if `valid_from` and `valid_to` ever overlap. The filtered unique index on open rows prevents the common version of that bug, and routing all reads through the

view prevents the rest.

In exchange you stop duplicating a dimension you don't own, you capture changes made by other teams, and you get a verifiable record of exactly what each model run was given. For an exercise whose whole purpose is to measure whether definition changes improved accuracy, that last point is worth the extra join.